Activation Functions in Neural Networks

What is an Activation Function?

An activation function determines whether a neuron should be activated or not. It introduces non-linearity into the neural network, allowing the model to learn complex patterns.

Without activation functions, a neural network behaves like a simple linear regression model regardless of how many layers it has.

Why Activation Functions are Important

Activation functions help neural networks:

- Learn complex patterns
- Handle non-linear data
- Improve prediction capability
- Enable deep learning architectures
- Control neuron output

Types of Activation Functions

1. Binary Step Function
2. Linear Activation Function
3. Sigmoid Function
4. Tanh Function
5. ReLU (Rectified Linear Unit)
6. Leaky ReLU
7. ELU (Exponential Linear Unit)
8. Softmax Function
9. Swish Function
10. GELU Function

---

1. Binary Step Function

Definition

The Binary Step function converts input into either 0 or 1.

If input >= 0 → output = 1 If input < 0 → output = 0

Formula

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

Advantages

- Simple to understand
- Used in early perceptrons

Disadvantages

- Not differentiable
- Gradient is zero almost everywhere
- Cannot be used effectively in deep learning

Python Implementation

```
import numpy as np


def step_function(x):
    return np.where(x >= 0, 1, 0)


x = np.array([-3, -1, 0, 2, 5])
print(step_function(x))
```

---

2. Linear Activation Function

Definition

Output is directly proportional to input.

Formula

$$f(x) = x$$

Advantages

- Simple

- Useful in regression output layers

Disadvantages

- Cannot learn complex patterns
- No non-linearity

Python Implementation

```
import numpy as np


def linear_activation(x):
    return x


x = np.array([-2, 0, 3])
print(linear_activation(x))
```

---

3. Sigmoid Activation Function

Definition

Sigmoid maps values between 0 and 1.

Commonly used in binary classification.

Formula

$$f(x) = \frac{1}{1 + e^{-x}}$$

Advantages

- Smooth curve
- Output interpretable as probability

Disadvantages

- Vanishing gradient problem
- Slow training
- Not zero-centered

Python Implementation

```python
import numpy as np


def sigmoid(x):
    return 1 / (1 + np.exp(-x))


x = np.array([-3, -1, 0, 2])
print(sigmoid(x))
```

---

4. Tanh Activation Function

Definition

Tanh outputs values between -1 and 1.

Formula

$$f(x) = \tanh(x)$$

Advantages

- Zero-centered
- Better than sigmoid in many cases

Disadvantages

- Still suffers from vanishing gradient

Python Implementation

```python
import numpy as np


def tanh_activation(x):
    return np.tanh(x)
```

```
x = np.array([-3, -1, 0, 2])
print(tanh_activation(x))
```

---

5. ReLU (Rectified Linear Unit)

Definition

ReLU returns input directly if positive, otherwise returns 0.

Formula

$$f(x) = \max(0, x)$$

Advantages

- Fast computation
- Reduces vanishing gradient problem
- Most widely used activation function

Disadvantages

- Dying ReLU problem

Python Implementation

```
import numpy as np


def relu(x):
    return np.maximum(0, x)


x = np.array([-3, -1, 0, 2])
print(relu(x))
```

---

6. Leaky ReLU

Definition

Leaky ReLU allows a small gradient for negative values.

Formula

$$f(x) = \begin{cases} x & x > 0 \\ 0.01x & x \leq 0 \end{cases}$$

Advantages

- Solves dying ReLU problem
- Better gradient flow

Disadvantages

- Small negative slope chosen manually

Python Implementation

```
import numpy as np


def leaky_relu(x, alpha=0.01):
    return np.where(x > 0, x, alpha * x)


x = np.array([-3, -1, 0, 2])
print(leaky_relu(x))
```

---

7. ELU (Exponential Linear Unit)

Definition

ELU smooths negative values instead of making them zero.

Formula

$$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

Advantages

- Faster learning
- Reduces bias shift

Disadvantages

- Computationally expensive

Python Implementation

```
import numpy as np


def elu(x, alpha=1.0):
    return np.where(x > 0, x, alpha * (np.exp(x) - 1))


x = np.array([-3, -1, 0, 2])
print(elu(x))
```

---

8. Softmax Function

Definition

Softmax converts values into probabilities.

Used in multi-class classification.

Formula

$$softmax(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Advantages

- Produces probability distribution
- Useful in classification tasks

Disadvantages

- Sensitive to large values

Python Implementation

```
import numpy as np


def softmax(x):
    exp_values = np.exp(x - np.max(x))
    return exp_values / np.sum(exp_values)


x = np.array([2.0, 1.0, 0.1])
print(softmax(x))
```

---

9. Swish Activation Function

Definition

Swish is developed by Google.

Formula

$$f(x) = x \cdot sigmoid(x)$$

Advantages

- Smooth
- Better performance in deep networks

Disadvantages

- More computationally expensive than ReLU

Python Implementation

```
import numpy as np


def sigmoid(x):
    return 1 / (1 + np.exp(-x))


def swish(x):
```

```
    return x * sigmoid(x)


x = np.array([-3, -1, 0, 2])
print(swish(x))
```

---

10. GELU (Gaussian Error Linear Unit)

Definition

GELU is commonly used in Transformers such as BERT and GPT.

Formula

$$GELU(x) = x\Phi(x)$$

Approximation:

$$0.5x\left(1+\tanh\left(\sqrt{\frac{2}{\pi}}(x+0.044715x^3)\right)\right)$$

Advantages

- Smooth activation
- Excellent performance in transformers

Disadvantages

- Computationally expensive

Python Implementation

```
import numpy as np


def gelu(x):
    return 0.5 * x * (
        1 + np.tanh(
            np.sqrt(2 / np.pi) * (x + 0.044715 * np.power(x, 3))
        )
    )
```

```
x = np.array([-3, -1, 0, 2])
print(gelu(x))
```

Comparison Table

Activation Function	Output Range	Differentiable	Common Usage
Step	0 or 1	No	Early perceptrons
Linear	(-∞, ∞)	Yes	Regression
Sigmoid	(0, 1)	Yes	Binary classification
Tanh	(-1, 1)	Yes	Hidden layers
ReLU	[0, ∞)	Yes	Deep learning
Leaky ReLU	(-∞, ∞)	Yes	Deep networks
ELU	(-α, ∞)	Yes	Advanced networks
Softmax	(0, 1)	Yes	Multi-class classification
Swish	(-∞, ∞)	Yes	Modern deep learning
GELU	(-∞, ∞)	Yes	Transformers

Which Activation Function Should You Use?

Hidden Layers

Recommended:

- ReLU
- Leaky ReLU
- GELU
- Swish

Output Layer

Problem Type	Activation Function
Binary Classification	Sigmoid

Problem Type	Activation Function
Multi-class Classification	Softmax
Regression	Linear

Vanishing Gradient Problem

This occurs when gradients become extremely small during backpropagation.

Common in:

- Sigmoid
- Tanh

Effects:

- Slow training
- Model stops learning

Solutions:

- ReLU
- Leaky ReLU
- Batch Normalization
- Residual Networks

Dying ReLU Problem

In ReLU, neurons can permanently output 0 for negative inputs.

Solution:

- Leaky ReLU
- ELU

TensorFlow / Keras Example

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(10,)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.summary()
```

PyTorch Example

```
import torch
import torch.nn as nn


model = nn.Sequential(
    nn.Linear(10, 64),
    nn.ReLU(),
    nn.Linear(64, 32),
    nn.ReLU(),
    nn.Linear(32, 1),
    nn.Sigmoid()
)

print(model)
```

Graph Visualization for Activation Functions

The following Python script plots graphs for all major activation functions using Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt


x = np.linspace(-10, 10, 1000)


# Step Function
step = np.where(x >= 0, 1, 0)
```

```
# Linear
linear = x

# Sigmoid
sigmoid = 1 / (1 + np.exp(-x))

# Tanh
tanh = np.tanh(x)

# ReLU
relu = np.maximum(0, x)

# Leaky ReLU
leaky_relu = np.where(x > 0, x, 0.01 * x)

# ELU
elu = np.where(x > 0, x, np.exp(x) - 1)

# Swish
swish = x * sigmoid

# GELU
gelu = 0.5 * x * (
    1 + np.tanh(
        np.sqrt(2 / np.pi) * (x + 0.044715 * np.power(x, 3))
    )
)


functions = {
    'Step Function': step,
    'Linear': linear,
    'Sigmoid': sigmoid,
    'Tanh': tanh,
    'ReLU': relu,
    'Leaky ReLU': leaky_relu,
    'ELU': elu,
    'Swish': swish,
    'GELU': gelu
}


for name, y in functions.items():
    plt.figure(figsize=(6, 4))
    plt.plot(x, y)
    plt.title(name)
    plt.xlabel('x')
```

```
    plt.ylabel('f(x)')
    plt.grid(True)
    plt.show()
```

Summary

Activation functions are essential in neural networks because they introduce non-linearity and enable deep learning models to learn complex patterns.

Most commonly used activation functions today:

- ReLU
- Leaky ReLU
- GELU
- Softmax
- Sigmoid

Selection depends on:

- Problem type
- Network architecture
- Training performance
- Computational efficiency